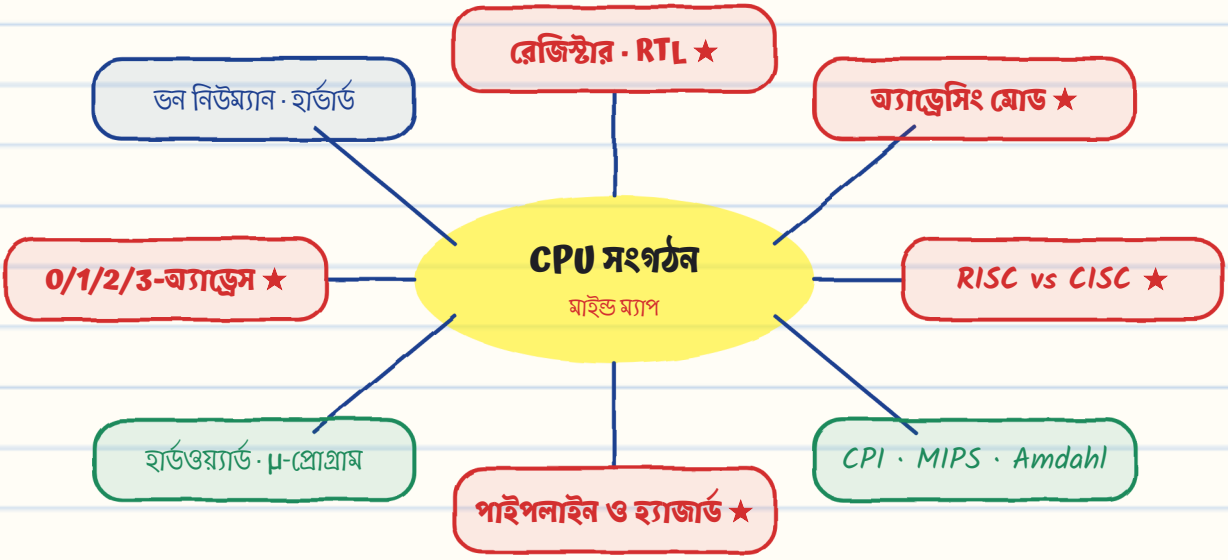
CPU সংগঠন (CPU Organization)

রেজিস্টার · RTL · অ্যাড্রেসিং · RISC/CISC · পাইপলাইন · পারফরম্যান্স: ১৪ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = পদ্ধতি / গাণিতিক

➜ সিলেবাসের যে অংশ

"Computer Architecture: ... Measuring performance; ... representing instruction, addressing styles; ALU operations, floating point operations, designing ALU; Processor design: datapaths & single cycle and multicycle implementations; Control Unit design: hardwired and microprogrammed; Hazards; Exceptions; Pipeline ..., superscalar and dynamic pipelining; ... channels." | ক্যাশ ও ভার্চুয়াল মেমরি আছে মেমরি হায়ারার্কি লেসনে।

হাই-ইল্ড তালিকা

① PC · IR · MAR · MDR
② RTL-এ ফেচ সাইকেল
③ প্রতিটি মোডের EA
④ k + n − 1 সাইকেল
⑤ RAW · ফরওয়ার্ডিং
⑥ IC × CPI / f · Amdahl

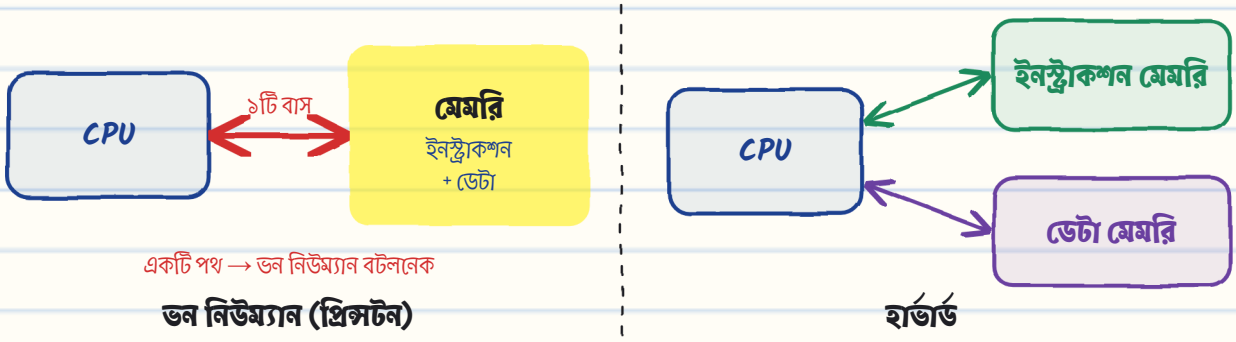
সংকেত হলুদ = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

➜ সূচিপত্র

✓ ভন নিউম্যান, হার্ভার্ড, CPU ..... পৃ. ২
✓ বাস, রেজিস্টার, ফ্ল্যাগ ..... পৃ. ৩
✓ ইনস্ট্রাকশন সাইকেল ও RTL ... পৃ. ৪
✓ ইনস্ট্রাকশন ফরম্যাট ..... পৃ. ৫
✓ অ্যাড্রেসিং মোড ..... পৃ. ৬
✓ RISC/CISC, কন্ট্রোল ইউনিট ... পৃ. ৭
✓ পাইপলাইন ও স্পিডআপ ..... পৃ. ৮
✓ হ্যাজার্ড ও প্রতিকার ..... পৃ. ৯
✓ পারফরম্যান্স ও Amdahl ..... পৃ. ১০
✓ ALU, ফিক্সড ও ফ্লোটিং পয়েন্ট ... পৃ. ১১
✓ সুপারস্কেলার, ফ্লিন, I/O ..... পৃ. ১২
✓ পরীক্ষার আগের রাতের পাতা ... পৃ. ১৩
✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১৪

১ সংরক্ষিত প্রোগ্রাম: ভন নিউম্যান বনাম হার্ভার্ড হাই-ইল্ড

সংরক্ষিত প্রোগ্রাম ধারণা (Stored-program concept) ইনস্ট্রাকশন থাকে ডেটার সাথে একই রিড-রাইট মেমরিতে, আর প্রোগ্রাম কাউন্টারের নিয়ন্ত্রণে একটির পর একটি ফেচ ও এক্সিকিউট হয়। জন ভন নিউম্যান, ১৯৪৫, *"First Draft of a Report on the EDVAC"*।



একটি শেয়ার্ড মেমরি + বাস (বামে) বনাম আলাদা ইনস্ট্রাকশন ও ডেটা পথ (ডানে)

বিষয়	ভন নিউম্যান	হার্ভার্ড
মেমরি	কোড ও ডেটার জন্য একটি মেমরি	কোড ও ডেটার আলাদা মেমরি
বাস	একটি শেয়ার্ড সেট	দুটি সেট: ফেচ + ডেটা **একসাথে**
গতি / খরচ	ধীর (বটলনেক), সস্তা	দ্রুত, বেশি ওয়্যারিং, ব্যয়বহুল
ব্যবহার	*PC (মেইন মেমরি), EDVAC, IAS*	*DSP, PIC/AVR MCU, Mark I (1944)*

পরীক্ষার ফাঁদ **ভন নিউম্যান বটলনেক** মানে **একটিমাত্র** পথে CPU ও মেমরির মধ্যে সীমিত ডেটা-স্থানান্তর হার (শব্দটি জনপ্রিয় করেন জন ব্যাকাস, ১৯৭৭); ধীর ALU নয়।

আধুনিক CPU = মডিফায়েড হার্ভার্ড: L1-এ আলাদা ইনস্ট্রাকশন ও ডেটা ক্যাশ, কিন্তু মেইন মেমরি একটিই (দেখো: মেমরি হায়ারার্কি)।

২ CPU-এর ভেতরে: ALU, CU, রেজিস্টার MCQ

- **ALU**: গাণিতিক (যোগ, বিয়োগ, ইনক্রিমেন্ট, তুলনা) ও লজিক (AND, OR, XOR, NOT, শিফট) কাজ করে; ফ্ল্যাগ সেট করে।
- **কন্ট্রোল ইউনিট (CU)**: ইনস্ট্রাকশন ফেচ ও **ডিকোড** করে এবং টাইমিং ও কন্ট্রোল সিগন্যাল পাঠায়, যাতে ALU, রেজিস্টার, মেমরি ও I/O কাজ করে। CU নির্দেশ দেয়; কাজটি করে ALU ও ডেটাপথ।
- **রেজিস্টার**: CPU-এর ভেতরের সবচেয়ে দ্রুত স্টোরেজ (পরের পাতা)। ALU + রেজিস্টার + অভ্যন্তরীণ বাস = ডেটাপথ (datapath)।

সিস্টেম বাস	দিক	প্রস্থ যা ঠিক করে
অ্যাড্রেস	একমুখী, CPU →	2^n লোকেশন
ডেটা	দ্বিমুখী	ওয়ার্ড সাইজ
কন্ট্রোল	মিশ্র লাইন	RD, WR, INT, clock

বাসের হিসাব ✍️ 16-বিট অ্যাড্রেস বাস (8085) → 2^{16} = **64 KB** · 20-বিট (8086) → **1 MB** · 32-বিট → **4 GB**। MAR-এর প্রস্থ = অ্যাড্রেস বাস; MDR-এর প্রস্থ = ডেটা বাস।

৩ CPU-এর রেজিস্টারসমূহ খুব হাই-ইল্ড

রেজি.	পূর্ণ নাম	কী রাখে / কী করে
PC	Program Counter (x86: IP)	পরবর্তী ইনস্ট্রাকশনের অ্যাড্রেস; প্রতি ফেচে +1
IR	Instruction Register	যে ইনস্ট্রাকশন **এখন ডিকোড / এক্সিকিউট** হচ্ছে
MAR	Memory Address Register	যে অ্যাড্রেস পড়া/লেখা হবে; অ্যাড্রেস বাস চালায়
MDR	Memory Data Reg. (MBR)	সদ্য পড়া, বা লিখতে যাওয়া ওয়ার্ড
AC	Accumulator	অন্তর্নিহিত অপারেন্ড + ALU-এর ফল (1-অ্যাড্রেস)
SP	Stack Pointer	**স্ট্যাকের শীর্ষ**: PUSH/POP, CALL/RET, ইন্টারাপ্ট
GPR	R0 … Rn	অপারেন্ড, অস্থায়ী মান (MIPS-এ 32টি)
PSW	ফ্ল্যাগ / স্ট্যাটাস রেজিস্টার	শেষ ফলের কন্ডিশন বিট + কন্ট্রোল বিট

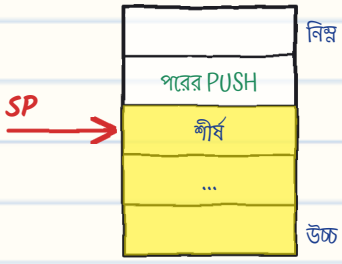
ফ্ল্যাগ	= 1 যখন
Z	ফল শূন্য
S/N	MSB = 1 (ঋণাত্মক)
C	MSB থেকে ক্যারি / বরো
V/O	সাইনড ওভারফ্লো
P	**জোড়** প্যারিটি (x86)
AC	বিট 3 → 4 ক্যারি (BCD)

8086 ফ্ল্যাগ রেজিস্টার ১৬ বিট, **৯টি ফ্ল্যাগ** ৬টি স্ট্যাটাস (CF, PF, AF, ZF, SF, OF) + ৩টি কন্ট্রোল (TF ট্র্যাপ, IF ইন্টারাপ্ট, DF ডিরেকশন)।

সমাধান ✍️ 8-বিট 7FH + 01H = 80H
Z = 0, S = 1, C = 0, **V = 1** (+127 + 1 ≠ −128), AC = 1, P = 0 (একটি 1-বিট)

স্ট্যাক (Stack, LIFO) বেশিরভাগ CPU-তে (x86) নিচের অ্যাড্রেসের দিকে বাড়ে:

PUSH: SP ← SP − 1, M[SP] ← DR
POP: DR ← M[SP], SP ← SP + 1

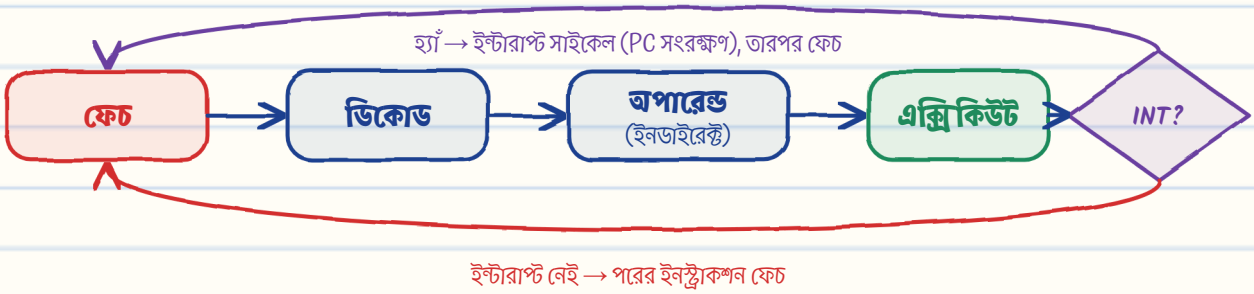


পাতায় ওপরে বাড়ে = নিচের অ্যাড্রেসে

মনে রাখার কৌশল "**PC দেখায়, IR ধরে, MAR ঠিকানা দেয়, MDR বয়ে আনে**।" ফেচ সবসময় PC → MAR → MDR → IR পথে চলে।

৪ ইনস্ট্রাকশন সাইকেল ও মাইক্রো-অপারেশন (RTL) লিখিতে প্রিয়

ইনস্ট্রাকশন সাইকেল ফেচ → ডিকোড → (অপারেন্ড ফেচ / ইনডাইরেক্ট) → এক্সিকিউট → ইন্টারাপ্ট যাচাই, HALT পর্যন্ত বারবার। প্রতিটি ধাপ কয়েকটি **মাইক্রো-অপারেশনের** (micro-operation) ধারা: এক ক্লক পালসে হওয়া মৌলিক রেজিস্টার ট্রান্সফার।



ধাপ	পালস	মাইক্রো-অপারেশন (RTL)	অর্থ
ফেচ	t1	**MAR ← PC**	অ্যাড্রেস বাইরে
	t2	**MDR ← M[MAR], PC ← PC + 1**	পড়া, PC + 1
	t3	**IR ← MDR**	IR লোড
ADD X	t1	MAR ← IR(address)	অপারেন্ডের অ্যাড্রেস
	t2	MDR ← M[MAR]	অপারেন্ড পড়া
	t3	AC ← AC + MDR	ALU যোগ করে
STORE X	t1–t2	MAR ← IR(addr), MDR ← AC; M[MAR] ← MDR	AC লেখা
JMP X	t1	PC ← IR(address)	মেমরি লাগে না
ইন্টারাপ্ট	t1–t3	MDR ← PC; MAR ← save, PC ← ISR; M[MAR] ← MDR	PC সংরক্ষণ

RTL চিহ্ন: ← ট্রান্সফার · কমা = একই ক্লক পালসে · P: R2 ← R1 = কেবল কন্ট্রোল ফাংশন P = 1 হলে · M[AR] = AR অ্যাড্রেসের মেমরি ওয়ার্ড।

চার ধরনের মাইক্রো-অপারেশন: রেজিস্টার ট্রান্সফার (R1 ← R2), গাণিতিক (R3 ← R1 + R2' + 1), লজিক (R1 ← R1 ⊕ R2), শিফট (R1 ← shl R1)।

ইনস্ট্রাকশনের ধরন	উদাহরণ
ডেটা ট্রান্সফার	MOV, LOAD, STORE
ডেটা প্রসেসিং	ADD, SUB, AND, SHL
প্রোগ্রাম কন্ট্রোল	JMP, JZ, CALL, RET
I/O	IN, OUT

পরীক্ষার ফাঁদ PC বাড়ে **ফেচের সময়েই**, তাই এক্সিকিউটের সময় PC-তে থাকে **পরের** ইনস্ট্রাকশনের অ্যাড্রেস। PC-রিলেটিভ অ্যাড্রেস সেখান থেকেই গোনা হয়।

মনে রাখার কৌশল ফেচ = "**৩টি T**": অ্যাড্রেস বাইরে, ডেটা ভেতরে (+ PC বাড়ে), IR-এ। প্রতিটি ইনস্ট্রাকশনকে এই ৩ পালস দিতেই হয়।

৫ ইনস্ট্রাকশন ফরম্যাট: ৩, ২, ১ ও ০ অ্যাড্রেস হাই-ইল্ড

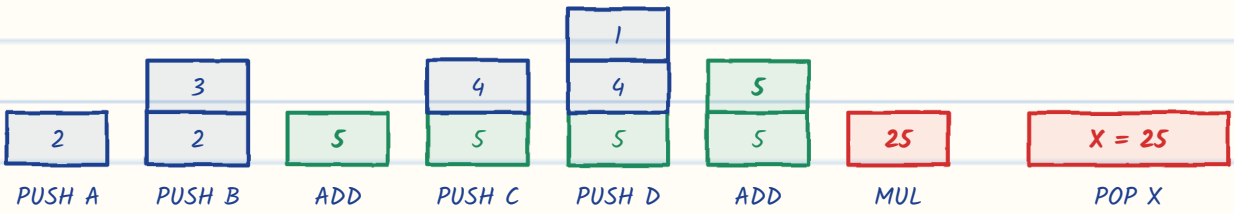
ফরম্যাট একটি ইনস্ট্রাকশন = অপকোড (opcode) (কী করতে হবে) + মোড বিট (অপারেন্ড কীভাবে খুঁজবে) + অ্যাড্রেস / রেজিস্টার ফিল্ড। n বিট অপকোড → সর্বোচ্চ 2^n টি অপারেশন।

→ চার ধরনের মেশিনে X = (A + B) × (C + D) নির্ণয়

৩-অ্যাড্রেস	২-অ্যাড্রেস	১-অ্যাড্রেস (অ্যাকুমুলেটর)	০-অ্যাড্রেস (স্ট্যাক)
ADD R1, A, B	MOV R1, A	LOAD A ; AC ← A	PUSH A
ADD R2, C, D	ADD R1, B	ADD B	PUSH B
MUL X, R1, R2	MOV R2, C	STORE T ; temp	ADD
	ADD R2, D	LOAD C	PUSH C
	MUL R1, R2	ADD D	PUSH D
	MOV X, R1	MUL T	ADD
		STORE X	MUL · POP X
৩টি	**৬টি**	**৭টি**	**৮টি**

০-অ্যাড্রেস কোড আসলে পোস্টফিক্স (RPN) রূপ AB+CD+× বাম থেকে ডানে পড়া: অপারেন্ড → PUSH, অপারেটর → দুটি POP করে ফল PUSH।

↙ অ্যাড্রেস কম → ইনস্ট্রাকশন ছোট, প্রোগ্রাম লম্বা



স্ট্যাকের ধাপ: A = 2, B = 3, C = 4, D = 1 → (2 + 3)(4 + 1) = 25

ফিল্ডের মাপ ✍ 32-বিট ইনস্ট্রাকশন, 64টি রেজিস্টার, 64K-ওয়ার্ড মেমরি, ফরম্যাট opcode | reg | address।
reg = $\log_2 64$ = 6 বিট · address = $\log_2 65536$ = 16 বিট
opcode = 32 − 6 − 16 = **10 বিট** → 1024টি অপারেশন

এক্সপান্ডিং অপকোড ✍ 32-বিট ইনস্ট্রাকশন, 12-বিট অ্যাড্রেস, ২৫০টি ২-অ্যাড্রেস ইনস্ট্রাকশন। কয়টি ১-অ্যাড্রেস ইনস্ট্রাকশন সম্ভব?
২-অ্যাড্রেস অপকোড = 32 − 24 = 8 বিট → 256টি প্যাটার্ন; 6টি খালি
প্রতিটি আরও 12 বিট পায়: 6 × 2^{12} = **24 576**

পরীক্ষার ফাঁদ "০-অ্যাড্রেস" মেশিনেও **PUSH/POP-এ একটি অ্যাড্রেস থাকে**; শুধু ALU ইনস্ট্রাকশনে (ADD, MUL) কোনো অ্যাড্রেস নেই।

মনে রাখার কৌশল ৩-২-১-০ → **৩-৬-৭-৮**টি ইনস্ট্রাকশন (এই রাশিতে): অ্যাড্রেস যত কম, ইনস্ট্রাকশন তত বেশি।

৬ অ্যাড্রেসিং মোড ও কার্যকর অ্যাড্রেস (EA) খুব হাই-ইল্ড

সংজ্ঞা অ্যাড্রেসিং মোড (Addressing mode) বলে দেয় ইনস্ট্রাকশনের অ্যাড্রেস ফিল্ড থেকে কীভাবে কার্যকর অ্যাড্রেস (Effective Address, EA) পাওয়া যাবে, অর্থাৎ অপারেন্ডের আসল মেমরি অ্যাড্রেস। মোডের কারণে ছোট ইনস্ট্রাকশনেই পয়েন্টার, অ্যারে, লুপ ও রিলোকেটেবল কোড সম্ভব।

মোড	অপারেন্ড / EA	উদাহরণ	সাধারণ ব্যবহার
ইমপ্লায়েড	অপকোডেই নির্দিষ্ট	CMA, CLC	অ্যাকুমুলেটর, ফ্ল্যাগ
ইমিডিয়েট	অপারেন্ড ইনস্ট্রাকশনের **ভেতরে**	MOV R1, #25	ধ্রুবক, সবচেয়ে দ্রুত
রেজিস্টার	অপারেন্ড রেজিস্টার R-এ	ADD R1, R2	মেমরি অ্যাক্সেস নেই
রেজিস্টার ইনডাইরেক্ট	EA = (R)	MOV R1, (R2)	পয়েন্টার
ডাইরেক্ট (অ্যাবসোলিউট)	EA = অ্যাড্রেস ফিল্ড A	LOAD 500	গ্লোবাল ভেরিয়েবল
ইনডাইরেক্ট	EA = M[A]	LOAD @500	মেমরিতে পয়েন্টার
রিলেটিভ	EA = PC + A	BEQ +12	ব্রাঞ্চ, রিলোকেশন
ইনডেক্সড	EA = A + (XR)	LOAD 500(X)	অ্যারে
বেস রেজিস্টার	EA = (BR) + A	[BX + 8]	রিলোকেশন, সেগমেন্ট
অটো-ইনক / ডেক	EA = (R), R ± 1 পরে / আগে	(R1)+, −(R1)	অ্যারে, স্ট্যাক

➜ সমাধানকৃত উদাহরণ (পরীক্ষার ক্লাসিক সেটিং)

PC = 200

R1 = 400

XR = 100

২-ওয়ার্ড ইনস্ট্রাকশন 200-201-এ

200	LOAD, mode
201	500
202	পরের ইনস্ট্রা.
399	450
400	700
500	800
600	900
702	325
800	300

মোড	EA	AC-তে যায়
ডাইরেক্ট	500	800
ইমিডিয়েট	201	**500**
ইনডাইরেক্ট	800	300
রিলেটিভ (PC = 202)	**702**	325
ইনডেক্সড (XR = 100)	600	900
রেজিস্টার (R1)	–	400
রেজিস্টার ইনডাইরেক্ট	400	700
অটোইনক্রিমেন্ট	400	700 (R1 → 401)
অটোডিক্রিমেন্ট	399	450 (R1 → 399)

মনে রাখার কৌশল অপারেন্ড পর্যন্ত "**লাফ**" গোনো: ইমিডিয়েট 0, ডাইরেক্ট 1, ইনডাইরেক্ট 2। রেজিস্টার মোড একটি মেমরি-লাফের বদলে রেজিস্টার ব্যবহার করে।

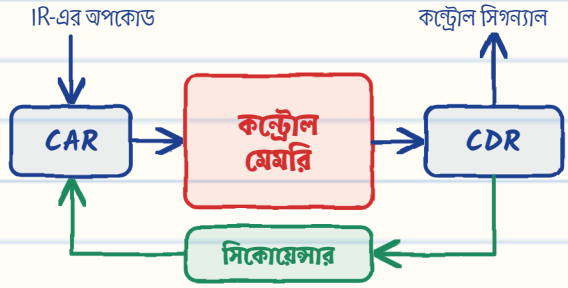
পরীক্ষার ফাঁদ রিলেটিভ মোডে যোগ হয় **হালনাগাদ** PC (এখানে 202), 200 নয়। ইমিডিয়েটের EA হলো অপারেন্ড ফিল্ডেরই অ্যাড্রেস (201)।

৭ RISC বনাম CISC লিখিতে প্রিয়

বিষয়	RISC (রিডিউসড)	CISC (কমপ্লেক্স)
ইনস্ট্রাকশন সেট	কম, সরল	অনেক, কিছু খুব জটিল
দৈর্ঘ্য	নির্দিষ্ট (32-বিট)	পরিবর্তনশীল (x86: 1–15 বাইট)
অ্যাড্রেসিং মোড	কম	অনেক
মেমরি অ্যাক্সেস	শুধু LOAD/STORE	ALU ইনস্ট্রাকশনেও মেমরি অপারেন্ড
সাইকেল / ইনস্ট্রা.	≈ 1, সহজে পাইপলাইন	অনেক, অসমান; পাইপলাইন কঠিন
কন্ট্রোল ইউনিট	হার্ডওয়্যার্ড	মাইক্রোপ্রোগ্রামড
রেজিস্টার	বড় রেজিস্টার ফাইল	কম GPR
কোড / কম্পাইলার	কোড লম্বা, কম্পাইলারের কাজ বেশি	কোড ছোট, হার্ডওয়্যারের কাজ বেশি
উদাহরণ	ARM, MIPS, RISC-V, SPARC	Intel x86, VAX, Motorola 68000

আধুনিক x86 চিপ ভেতরে CISC ইনস্ট্রাকশনকে RISC-ধাঁচের **মাইক্রো-অপ (micro-op)**-এ ভেঙে নেয়, তাই সীমারেখা ঝাপসা। RISC-এর সূচনা IBM 801 (১৯৭৫), বার্কলে RISC ও স্ট্যানফোর্ড MIPS (১৯৮০-র দশকের শুরু) থেকে।

৮ কন্ট্রোল ইউনিট: হার্ডওয়্যার্ড বনাম মাইক্রোপ্রোগ্রামড হাই-ইল্ড



মাইক্রোপ্রোগ্রামড CU (উইলকস, ১৯৫১)

হার্ডওয়্যার্ড (Hardwired): কন্ট্রোল সিগন্যাল আসে একটি স্থির লজিক সার্কিট থেকে: সিকোয়েন্স কাউন্টার + ডিকোডার + গেট (বা PLA/FSM)।

মাইক্রোপ্রোগ্রামড (Microprogrammed): প্রতিটি মেশিন ইনস্ট্রাকশন **কন্ট্রোল মেমরিতে** (ROM) রাখা মাইক্রোইনস্ট্রাকশনের একটি ছোট মাইক্রোপ্রোগ্রাম চালায়; CAR = কন্ট্রোল অ্যাড্রেস রেজিস্টার, CDR = কন্ট্রোল ডেটা রেজিস্টার।

বিষয়	হার্ডওয়্যার্ড	মাইক্রোপ্রোগ্রামড
গতি	দ্রুততর	ধীর (প্রতি ধাপে কন্ট্রোল মেমরি পড়া)
পরিবর্তন / ডিবাগ	সার্কিট নতুন করে নকশা	মাইক্রোকোড বদলালেই: **নমনীয়**
জটিল ISA	কঠিন, ব্যয়বহুল	সহজ
ব্যবহার	RISC	CISC

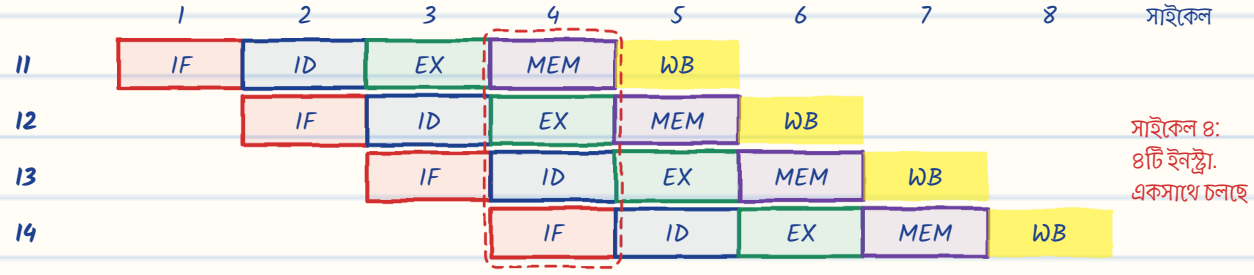
মাইক্রোইনস্ট্রাকশনের ধরন **হরাইজন্টাল:** প্রতি কন্ট্রোল সিগন্যালে ১ বিট, চওড়া, বেশি সমান্তরালতা, ডিকোড লাগে না। **ভার্টিক্যাল:** এনকোডেড ফিল্ড, সরু, ডিকোডার লাগে, ধীর।

ডেটাপাথ নকশা **সিঙ্গেল-সাইকেল:** CPI = 1, কিন্তু ক্লক = সবচেয়ে ধীর ইনস্ট্রাকশন (load)। **মাল্টিসাইকেল:** ছোট ক্লক, CPI 3–5, একই ইউনিট বিভিন্ন ধাপে পুনর্ব্যবহার।

মনে রাখার কৌশল **"হার্ড = হুড়মুড় (দ্রুত), মাইক্রো = মডিফাই":** হার্ডওয়্যার্ড দ্রুত কিন্তু অনমনীয়; মাইক্রোপ্রোগ্রামড ধীর কিন্তু বদলানো যায়।

৯ ইনস্ট্রাকশন পাইপলাইনিং (Pipelining) খুব হাই-ইল্ড

সংজ্ঞা ইনস্ট্রাকশন এক্সিকিউশনকে k-টি ধাপে (stage) ভাগ করে ধাপগুলোকে ওভারল্যাপ করা, অ্যাসেম্বলি লাইনের মতো: একটি এক্সিকিউট হওয়ার সময় পরেরটি ডিকোড এবং তার পরেরটি ফেচ হয়। ক্লাসিক RISC পাইপলাইনে ৫টি ধাপ: **IF** ফেচ, **ID** ডিকোড + রেজিস্টার পড়া, **EX** ALU, **MEM** মেমরি অ্যাক্সেস, **WB** রাইট ব্যাক।



স্পেস-টাইম ডায়াগ্রাম: ৪টি ইনস্ট্রাকশন ২০ নয়, ৫ + ৩ = ৮ সাইকেলে শেষ

সূত্র (k ধাপ, n ইনস্ট্রাকশন, ক্লক τ)

পাইপলাইনে সময় = $(k + n - 1)\,\tau$ — পাইপলাইন ছাড়া = $n \cdot t_n$ (সুষম হলে $t_n = k\,\tau$)

স্পিডআপ $S = n k / (k + n - 1) \to k$ — দক্ষতা $\eta = n / (k + n - 1) = S / k$

থ্রুপুট $= n / ((k + n - 1)\,\tau) \to 1/\tau$ — τ = ধীরতম ধাপ + ল্যাচ বিলম্ব

উদাহরণ ১ $k = 5$, $n = 100$, প্রতি ধাপে এক সাইকেল, স্টল নেই → $5 + 99 =$ **104 সাইকেল**; পাইপলাইন ছাড়া 500 সাইকেল; $S = 500 / 104 \approx$ **4.81**; $\eta = 100 / 104 \approx 96\%$।

উদাহরণ ২: অসমান ধাপ ধাপের বিলম্ব 60, 50, 90, 80 ns; ল্যাচ 10 ns; $n = 1000$। $\tau = 90 + 10 = 100$ ns · পাইপলাইনে $= (4 + 999) \times 100 = 100\,300$ ns · পাইপলাইন ছাড়া $= 1000 \times (60 + 50 + 90 + 80) = 280\,000$ ns · $S \approx$ **2.79**, ধাপ অসমান বলে 4-এর অনেক নিচে।

উদাহরণ ৩ IF 200, ID 100, EX 200, MEM 200, WB 100 ps। সিঙ্গেল-সাইকেল ক্লক = 800 ps (load পাঁচটি ধাপই ব্যবহার করে); পাইপলাইন ক্লক = 200 ps → আদর্শ স্পিডআপ 800 / 200 = **4**, S নয়।

পরীক্ষার ফাঁদ পাইপলাইন **থ্রুপুট** বাড়ায়; একটি ইনস্ট্রাকশনের ল্যাটেন্সি (latency) কমায় **না** (ল্যাচের কারণে বরং সামান্য বাড়ে)।

মনে রাখার কৌশল "**একবার ভরাও, তারপর প্রতি টিকে একটি**": প্রথম ফলের জন্য k সাইকেল, তারপর প্রতিটিতে আর 1 → $k + n - 1$।

পাইপলাইন রেজিস্টার (IF/ID, ID/EX, EX/MEM, MEM/WB) ধাপগুলোর মাঝে থেকে প্রতিটি ইনস্ট্রাকশনের ডেটা ও কন্ট্রোল বিট ধরে রাখে; কন্ট্রোল সিগন্যাল ID-তে তৈরি হয়ে ইনস্ট্রাকশনের সাথে পাইপ ধরে এগোয়।

১০ পাইপলাইন হ্যাজার্ড ও প্রতিকার

খুব হাই-ইল্ড

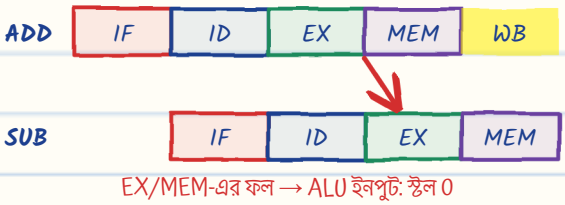
হ্যাজার্ড (Hazard) যে অবস্থার কারণে পরের ইনস্ট্রাকশন সময়মতো তার ধাপে ঢুকতে পারে না, ফলে স্টল (বাবল) দিতে হয়। বাস্তব CPI = 1 + প্রতি ইনস্ট্রাকশনে স্টল সাইকেল।

হ্যাজার্ড	কারণ	উদাহরণ	প্রতিকার
স্ট্রাকচারাল	দুটি ইনস্ট্রা.-র একই ইউনিট দরকার	একটি মেমরি পোর্ট	আলাদা I/D ক্যাশ, স্টল
ডেটা	অসমাপ্ত ফল দরকার	ADD R1,R2,R3 → SUB R4,R1,R5	ফরওয়ার্ডিং, স্টল
কন্ট্রোল	পরের PC অজানা	BEQ, JMP, CALL	ব্রাঞ্চ প্রেডিকশন

ডেটা হ্যাজার্ডের ধরন (i আগে, j পরে):

- **RAW** রিড আফটার রাইট: প্রকৃত নির্ভরতা, সবচেয়ে সাধারণ
- **WAR** রাইট আফটার রিড: অ্যান্টি-ডিপেন্ডেন্স
- **WAW** রাইট আফটার রাইট: আউটপুট ডিপেন্ডেন্স

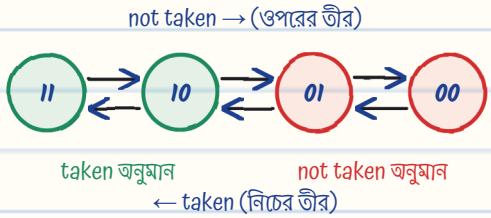
WAR ও WAW শুধু আউট-অফ-অর্ডার এক্সিকিউশনে ঘটে; **রেজিস্টার রিনেমিং** এগুলো দূর করে।



ফরওয়ার্ডিং (বাইপাসিং)

পরীক্ষার ফাঁদ: লোড-ইউজ LW R1, 0(R2)-এর ঠিক পরে ADD R3, R1, R4 থাকলে ফরওয়ার্ডিং থাকলেও **১টি স্টল** লাগে: লোড করা মান MEM-এর পরেই পাওয়া যায়। ফরওয়ার্ডিং না থাকলে পরের ইনস্ট্রাকশনে RAW-তে ২টি স্টল (WB-এর প্রথম অর্ধে রেজিস্টার ফাইলে লেখা, দ্বিতীয় অর্ধে পড়া)।

কন্ট্রোল হ্যাজার্ডের প্রতিকার: স্ট্যাটিক প্রেডিকশন (সবসময় not-taken; পেছনে taken / সামনে not-taken), **ডায়নামিক** প্রেডিকশন: প্রতি ব্রাঞ্চে ১-বিট বা ২-বিট স্যাচুরেটিং কাউন্টার + ব্রাঞ্চ টার্গেট বাফার (BTB), **ডিলেইড ব্রাঞ্চ** (ব্রাঞ্চের পরের স্লটে কম্পাইলার একটি কাজের ইনস্ট্রাকশন বসায়), ID-তেই ব্রাঞ্চ নিষ্পত্তি, ভুলভাবে ফেচ হওয়া ইনস্ট্রাকশন ফ্লাশ।



২-বিট প্রেডিক্টর: পরপর দুবার ভুলে তবে বদলায়

সমাধান: স্টলসহ CPI ৫-ধাপ পাইপলাইন, ২০% ব্রাঞ্চ (২-সাইকেল পেনাল্টি), ৩০% লোড যার ২৫%-এর পরেই ব্যবহার (১টি স্টল)।

CPI = 1 + 0.2 × 2 + 0.3 × 0.25 × 1 = **1.475** → পাইপলাইনবিহীনের তুলনায় স্পিডআপ ≈ 5 / 1.475 ≈ **3.39**।

৯০% সঠিক প্রেডিক্টর ও ৩-সাইকেল মিসপ্রেডিক্ট পেনাল্টিতে: CPI = 1 + 0.2 × 0.1 × 3 = **1.06**।

মনে রাখার কৌশল হ্যাজার্ড = "**SDC**": **S**hare (একই ইউনিট ভাগাভাগি), **D**epend (ডেটার ওপর নির্ভরতা), **C**hange (PC বদল)। একই ক্রমে প্রতিকার: ডুপ্লিকেট, ফরওয়ার্ড, প্রেডিক্ট।

১১ পারফরম্যান্স পরিমাপ গাণিতিক

CPU পারফরম্যান্স সমীকরণ

CPU time = IC × CPI × T = IC × CPI / f (IC = ইনস্ট্রাকশন সংখ্যা, T = 1/f)

গড় CPI = Σ (ভগ্নাংশ$_i$ × CPI$_i$) · **MIPS = f / (CPI × 10⁶)** = IC / (সময় × 10⁶) · পারফরম্যান্স = ১ / সময় · স্পিডআপ = পুরনো সময় / নতুন সময়

উপাদান	IC	CPI	ক্লক রেট f
নির্ভর করে	প্রোগ্রাম, কম্পাইলার, ISA	ISA, পাইপলাইন, ক্যাশ	প্রযুক্তি, সংগঠন

উদাহরণ ১ ✍️ মিশ্রণ: ৫০% ALU (CPI 1), ৩০% load/store (CPI 2), ২০% ব্রাঞ্চ (CPI 3)।
CPI = 0.5 + 0.6 + 0.6 = **1.7**
IC = 5 × 10⁸, f = 2 GHz → সময় = 5×10⁸ × 1.7 / 2×10⁹ = **0.425 s**; MIPS = 2000 / 1.7 ≈ **1176**

উদাহরণ ২ ✍️ একই প্রোগ্রাম। A: 2 GHz, CPI 2.0। B: 3 GHz, CPI 3.6।
A: IC × 2 / 2×10⁹ = 1.0 IC ns
B: IC × 3.6 / 3×10⁹ = 1.2 IC ns
ক্লক কম হলেও A **1.2 গুণ দ্রুত**।

উদাহরণ ৩: MIPS ✍️ 400 MHz, CPI 2 → MIPS = 400 × 10⁶ / (2 × 10⁶) = **200**। 400 MHz-এর ক্লক পিরিয়ড = 1 / (4 × 10⁸) = 2.5 ns।

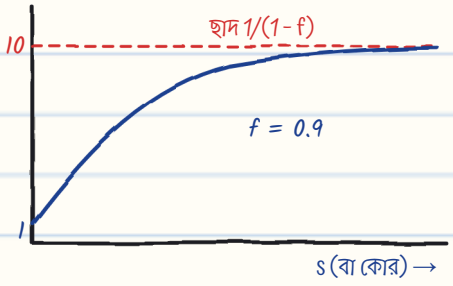
পরীক্ষার ফাঁদ শুধু ক্লক রেট বা শুধু MIPS দিয়ে পারফরম্যান্স মাপা যায় না: একটি ইনস্ট্রাকশন কতটা কাজ করে MIPS তা ধরে না (RISC বনাম CISC তুলনা করো)। ন্যায্য মাপ কেবল বাস্তব প্রোগ্রামের **এক্সিকিউশন সময়** (যেমন SPEC বেঞ্চমার্ক)।

১২ অ্যামডালের সূত্র (Amdahl's Law) হাই-ইল্ড

সময়ের **f** ভগ্নাংশ **s** গুণ দ্রুত করলে:

S = 1 / ((1 − f) + f / s) → সর্বোচ্চ **1 / (1 − f)**

- f = 0.4, s = 10 → 1 / (0.6 + 0.04) = **1.56**
- ৯০% সমান্তরাল, ১০টি কোর → 1 / (0.1 + 0.09) = **5.26** (সীমা 10)
- f = 0.8 দিয়ে মোট 2× চাই: 0.2 + 0.8/s = 0.5 → s = **2.67**



সিরিয়াল অংশই লাভের সীমা বেঁধে দেয়

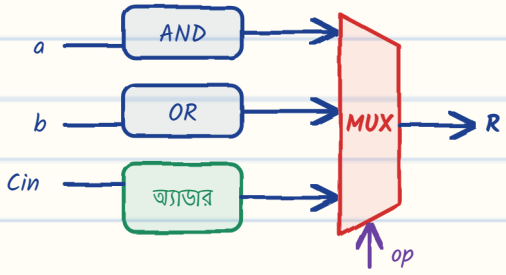
মনে রাখার কৌশল "**সাধারণ ঘটনাটিকে দ্রুত করো**": যে অংশ উন্নত করোনি (1 − f), সেটিই ছাদ ঠিক করে।

পরীক্ষার ফাঁদ f হলো **সময়ের** ভগ্নাংশ, ইনস্ট্রাকশনের নয়। f = 0.5 হলে s = ∞ দিলেও মাত্র 2×।

১৩ ALU নকশা ও ফিক্সড-পয়েন্ট গণিত

১-বিট ALU স্লাইস একসাথে AND, OR ও ফুল-অ্যাডারের যোগফল বের করে; অপারেশন-সিলেক্ট বিট অনুযায়ী একটি মাল্টিপ্লেক্সার একটিকে বেছে নেয়। ক্যারি জুড়ে ৩২টি স্লাইস = ৩২-বিট **রিপল-ক্যারি** ALU; **ক্যারি-লুকঅ্যাহেড** অ্যাডার ধীর ক্যারি-শৃঙ্খল এড়ায়।

বিয়োগ: $A - B = A + B' + 1$ (B উল্টাও, carry-in = 1)।



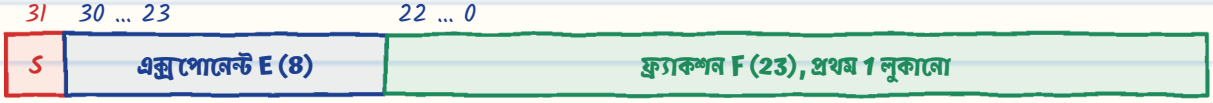
১-বিট ALU স্লাইস

n-বিট টু'স কমপ্লিমেন্ট পরিসর $-2^{n-1} \dots 2^{n-1} - 1$ (8-বিট: −128 … +127)। ওভারফ্লো $V = C_{in}(MSB) \oplus C_{out}(MSB)$: একই চিহ্নের দুটি সংখ্যার যোগফল উল্টো চিহ্নের হলে।

ওভারফ্লো ✍ 0110 0100 (+100) + 0100 0000 (+64) = 1010 0100 = 8 বিটে **−92**। MSB-তে ক্যারি ঢোকে 1, বের হয় 0 → **V = 1**।

গুণ: শিফট-অ্যান্ড-অ্যাড; **বুথের অ্যালগরিদম (Booth's algorithm)** সাইনড (টু'স কমপ্লিমেন্ট) অপারেন্ড সামলায় এবং টানা 1-গুলো এড়িয়ে যায় (জোড়া 10 → বিয়োগ, 01 → যোগ)। **ভাগ**: রিস্টোরিং / নন-রিস্টোরিং।

১৪ ফ্লোটিং পয়েন্ট (IEEE 754) হাই-ইল্ড



মান = $(-1)^s \times 1.F \times 2^{(E-127)}$

ফরম্যাট	সাইন	এক্সপোনেন্ট	ফ্র্যাকশন	বায়াস	নির্ভুলতা
সিঙ্গেল (32-বিট)	1	8	23	127	≈ 7 অঙ্ক, সর্বোচ্চ ≈ 3.4×10^{38}
ডাবল (64-বিট)	1	11	52	1023	≈ 15–16 অঙ্ক

সমাধান: সিঙ্গেল প্রিসিশনে −6.75 ✍ $6.75 = 110.11_2 = 1.1011 \times 2^2$ · সাইন = 1 · এক্সপোনেন্ট = 2 + 127 = 129 = 1000 0001 · ফ্র্যাকশন = 1011 000…0

1 | 10000001 | 10110000000000000000000 = **C0D80000H**

FP যোগ/বিয়োগ: ① মেলাও: ছোট এক্সপোনেন্টের ম্যান্টিসা ডানে শিফট ② ম্যান্টিসা যোগ/বিয়োগ ③ নরমালাইজ ④ রাউন্ড (গার্ড, রাউন্ড, স্টিকি বিট)।

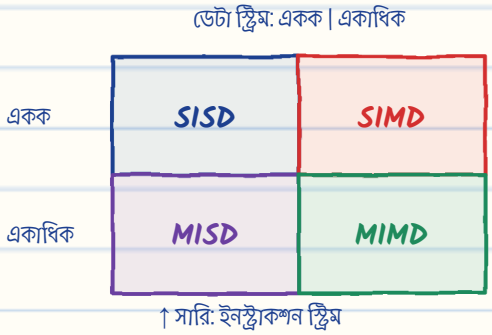
পরীক্ষার ফাঁদ এক্সপোনেন্ট সব 0 = শূন্য বা ডিনরমাল; সব 1 = ∞ ($F = 0$) বা NaN ($F \neq 0$)। সংরক্ষিত এক্সপোনেন্ট বায়াসড: 2^2 রাখা হয় 129 হিসেবে, 2 নয়।

মনে রাখার কৌশল "1-8-23, বায়াস 127" ও "1-11-52, বায়াস 1023": বায়াস = 2^(এক্সপোনেন্ট বিট − 1) − 1।

১৫ সাধারণ পাইপলাইনের পরে

- **সুপারস্কেলার (Superscalar)**: একাধিক পাইপলাইন; প্রতি সাইকেলে ২+ ইনস্ট্রাকশন ইস্যু, তাই CPI < 1 (IPC > 1)। **সুপারপাইপলাইনিং**: আরও বেশি, ছোট ধাপ। **VLIW**: কম্পাইলার সমান্তরাল অপারেশনগুলো একটি লম্বা ওয়ার্ডে বসায়।
- **ডায়নামিক পাইপলাইনিং** = আউট-অফ-অর্ডার এক্সিকিউশন: রিজার্ভেশন স্টেশন (টোমাসুলো), রেজিস্টার রিনেমিং, ইন-অর্ডার কমিটের জন্য রিঅর্ডার বাফার।
- **এক্সেপশন (Exception)** (ওভারফ্লো, অবৈধ অপকোড, পেজ ফল্ট) ঘটে CPU-এর ভেতরে; **ইন্টারাপ্ট** আসে I/O থেকে। প্রিসাইস এক্সেপশনে ত্রুটিপূর্ণ PC (EPC) ও কারণ সংরক্ষিত হয় এবং পরের ইনস্ট্রাকশনগুলো ফ্লাশ হয়।

১৬ ফ্লিনের শ্রেণিবিভাগ (Flynn's Taxonomy, ১৯৬৬) MCQ

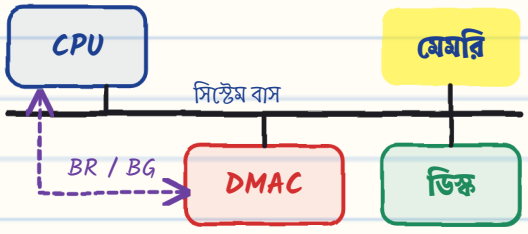


শ্রেণি	উদাহরণ
SISD	ক্লাসিক একক প্রসেসর (ভন নিউম্যান)
SIMD	ভেক্টর ও অ্যারে প্রসেসর, GPU, SSE/AVX
MISD	বিরল: রিডান্ড্যান্ট ফল্ট-টলারেন্ট সিস্টেম
MIMD	মাল্টিকোর CPU, SMP, ক্লাস্টার

১৭ I/O সংগঠন MCQ

পদ্ধতি	কীভাবে	CPU-এর ভূমিকা
প্রোগ্রামড I/O	CPU স্ট্যাটাস ফ্ল্যাগ পোল করে (busy-wait)	প্রতি ওয়ার্ডে, সাইকেল নষ্ট
ইন্টারাপ্ট-চালিত	ডিভাইস INT দেয়; CPU ISR চালায়	প্রতি ওয়ার্ডে, পোলিং নেই
DMA	DMA কন্ট্রোলার পুরো ব্লক সরায়	শুধু শুরু ও শেষে
IOP / চ্যানেল	I/O প্রসেসর চ্যানেল প্রোগ্রাম চালায়	সবচেয়ে কম

১. CPU DMA কন্ট্রোলারে শুরুর অ্যাড্রেস, ওয়ার্ড সংখ্যা ও দিক লেখে
২. DMAC **BR** (8086: HOLD) দেয়; CPU বাস সাইকেল শেষ করে **BG** (HLDA) দেয়
৩. DMAC সরাসরি মেমরি ↔ ডিভাইস ডেটা সরায়
৪. গণনা = 0 → DMAC বাস ছেড়ে দিয়ে CPU-কে ইন্টারাপ্ট করে



পরীক্ষার ফাঁদ DMA **সাইকেল স্টিলিং**-এ একবারে এক ওয়ার্ডের জন্য বাস নেয়; **বার্স্ট মোডে** পুরো ব্লকের জন্য ধরে রাখে। দুই ক্ষেত্রেই ডেটা CPU-এর রেজিস্টার দিয়ে যায় না। চ্যানেল: **সিলেক্টর** (একবারে একটি দ্রুত ডিভাইস) বনাম **মাল্টিপ্লেক্সার** (অনেক ধীর ডিভাইস পালাক্রমে)।

পরীক্ষার আগের রাতের পাতা

তথ্য-তালিকা

ফেচ: MAR←PC; MDR←M[MAR], PC+1; IR←MDR
PC পরের · IR বর্তমান · MAR ঠিকানা · MDR ডেটা
X = (A+B)(C+D): ৩/৬/৭/৮টি ইনস্ট্রাকশন
ডাইরেক্ট EA = A · ইনডাইরেক্ট EA = M[A]
পাইপলাইন: (k + n − 1) সাইকেল
τ = ধীরতম ধাপ + ল্যাচ
সময় = IC × CPI / f
Amdahl S = 1/((1 − f) + f/s)
IEEE সিঙ্গেল 1/8/23, বায়াস 127

অ্যাড্রেস বাস n বিট → 2^n লোকেশন
8086: ৯টি ফ্ল্যাগ (৬ স্ট্যাটাস + ৩ কন্ট্রোল)
n বিট অপকোড → 2^nটি অপারেশন
রিলেটিভ = নতুন PC + A · ইনডেক্সড = A + XR
S = nk/(k + n − 1) → k · η = S/k
বাস্তব CPI = 1 + প্রতি ইনস্ট্রা. স্টল
MIPS = f / (CPI × 10^6)
সীমা 1/(1 − f)
ডাবল 1/11/52, বায়াস 1023

→ MCQ-র জন্য সেরা ১৪টি এক-লাইনার

1. সংরক্ষিত প্রোগ্রাম: ভন নিউম্যান, ১৯৪৫
2. আলাদা I ও D মেমরি = হার্ভার্ড
3. পরের ইনস্ট্রাকশনের অ্যাড্রেস → PC
4. অপারেন্ড ইনস্ট্রাকশনেই → ইমিডিয়েট
5. অ্যারে অ্যাক্সেস → ইনডেক্সড মোড
6. স্ট্যাক মেশিনের ADD = 0-অ্যাড্রেস
7. RISC: নির্দিষ্ট দৈর্ঘ্য, load/store
8. কন্ট্রোল মেমরি → মাইক্রোপ্রোগ্রামড
9. হার্ডওয়্যার্ড CU: দ্রুত, কম নমনীয়
10. RAW = ডেটা হ্যাজার্ড → ফরওয়ার্ডিং
11. ব্রাঞ্চ → কন্ট্রোল হ্যাজার্ড → প্রেডিকশন
12. লোড-ইউজ: ফরওয়ার্ডিংয়েও ১টি স্টল
13. GPU / ভেক্টর = SIMD; মাল্টিকোর = MIMD
14. DMA: CPU শুধু শুরু ও শেষে

প্রধান ফাঁদ CU ডিকোড করে ও নির্দেশ দেয়, নিজে "এক্সিকিউট" করে না · ক্যাশ মেমরির একটি স্তর, CPU-এর ইউনিট নয় · রিলেটিভ মোডে হালনাগাদ PC · ইমিডিয়েটের EA = অপারেন্ড ফিল্ডের অ্যাড্রেস · পাইপলাইন থ্রুপুট বাড়ায়, ল্যাটেন্সি কমায় না · ধাপ অসমান হলে স্পিডআপ < k · MIPS ও GHz বিভ্রান্ত করে · 7FH + 01H-এ V সেট হয়, C নয়

সব কৌশল একসাথে PC দেখায়, IR ধরে, MAR ঠিকানা দেয়, MDR বয়ে আনে · ফেচ = ৩টি T · ৩-২-১-০ → ৩-৬-৭-৮ · লাফ গোনো · হার্ড = হুড়মুড়, মাইক্রো = মডিফাই · একবার ভরাও, তারপর প্রতি টিকে একটি · হ্যাজার্ড SDC → ডুপ্লিকেট, ফরওয়ার্ড, প্রেডিক্ট · সাধারণ ঘটনাটিকে দ্রুত করো · 1-8-23 বায়াস 127



t2-তে PC ← PC + 1, তারপর ডিকোড → এক্সিকিউট → আবার

যে ফেচ পথটি RTL-এ লিখতে পারতেই হবে

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

আগে নিজে করো, পরে মেলাও

১. ভন নিউম্যান মেশিনে ইনস্ট্রাকশন ও ডেটা থাকে:
ক) দুটি মেমরিতে খ) একটি মেমরিতে
গ) শুধু ROM-এ ঘ) রেজিস্টারে

২. ডিকোড হতে থাকা ইনস্ট্রাকশন কোন রেজিস্টারে থাকে?
ক) PC খ) MAR
গ) IR ঘ) SP

৩. MOV R1, #25 কোন অ্যাড্রেসিং মোড?
ক) ডাইরেক্ট খ) ইনডাইরেক্ট
গ) ইনডেক্সড ঘ) ইমিডিয়েট

৪. ৫-ধাপ পাইপলাইন, ১০০টি ইনস্ট্রাকশন, স্টল নেই। সাইকেল?
ক) 104 খ) 500
গ) 105 ঘ) 100

৫. স্ট্যাক মেশিনের ADD-এ অ্যাড্রেস ফিল্ড কয়টি?
ক) 1 খ) 0
গ) 2 ঘ) 3

৬. 2 GHz CPU, গড় CPI 4 হলে:
ক) 800 MIPS খ) 8000 MIPS
গ) 500 MIPS ঘ) 200 MIPS

৭. রান টাইমের অর্ধেক 2× দ্রুত করা হলো। মোট স্পিডআপ:
ক) 2 খ) 1.5
গ) 1.25 ঘ) 1.33

৮. অপারেন্ড ফরওয়ার্ডিং মূলত কমায়:
ক) ডেটা হ্যাজার্ড খ) স্ট্রাকচারাল
গ) কন্ট্রোল হ্যাজার্ড ঘ) ক্যাশ মিস

৯. GPU / ভেক্টর প্রসেসর ফ্লিনের কোন শ্রেণিতে?
ক) SISD খ) SIMD
গ) MISD ঘ) MIMD

১০. DMA-তে CPU যুক্ত থাকে:
ক) প্রতি ওয়ার্ডে খ) কখনো না
গ) শুরু ও শেষে ঘ) শুধু ত্রুটিতে

উত্তরমালা → ১-খ · ২-গ · ৩-ঘ · ৪-ক · ৫-খ · ৬-গ · ৭-ঘ · ৮-ক · ৯-খ · ১০-গ

লিখিত প্রশ্ন একটি ৫-ধাপ পাইপলাইনের ধাপ-বিলম্ব 150, 120, 160, 140, 130 ps এবং ল্যাচ বিলম্ব 20 ps। ১০০০টি ইনস্ট্রাকশনের জন্য নির্ণয় করো (ক) ক্লক পিরিয়ড, (খ) পাইপলাইনসহ ও পাইপলাইন ছাড়া সময়, (গ) স্পিডআপ, (ঘ) ২০% ইনস্ট্রাকশন ২-সাইকেল পেনাল্টির ব্রাঞ্চ হলে সময়। ৮ নম্বর

১ τ = ধীরতম ধাপ + ল্যাচ = 160 + 20 = **180 ps** ১
২ পাইপলাইনে: (5 + 999) × 180 = **180 720 ps**; পাইপলাইন ছাড়া: 1000 × 700 = **700 000 ps** ৩
৩ স্পিডআপ = 700 000 / 180 720 ≈ **3.87** (সীমা 3.89, S নয়: ধাপ অসমান) ২
৪ স্টল = 1000 × 0.2 × 2 = 400 → (1004 + 400) × 180 = **252 720 ps**, S ≈ 2.77 ২

→ পরীক্ষার আগে যাচাই ✓

✓ ফেচ, ADD, ইন্টারাপ্ট সাইকেল RTL-এ
✓ প্রতিটি মোডে EA ও অপারেন্ড
✓ (A+B)(C+D) ৩/২/১/০-অ্যাড্রেস কোডে
✓ পাইপলাইন, CPI, MIPS, Amdahl অঙ্ক

লিখিতের জন্য আরও অনুশীলন:
① হার্ডওয়্যার্ড বনাম মাইক্রোপ্রোগ্রামড
② RISC বনাম CISC, ছয়টি পার্থক্য
③ তিন হ্যাজার্ড ও প্রতিকার
④ IEEE 754-এ 12.375